

8.1 STRATÉGIE : FORK ET RENOMMAGE VS REFONTE COMPLÈTE

Quand vous recevez une capsule Boutique ERP et que votre client est une école, une clinique, ou une coopérative, vous avez deux stratégies :

STRATÉGIE A – RENOMMAGE DE DOMAINE (rapide, 1-3 jours) :

Garder toute l'architecture. Renommer les concepts.

"Produit" devient "Élève", "Vente" devient "Inscription", etc.

IDÉAL SI : Les structures de données sont similaires.

RISQUE : Renommer sans comprendre peut casser les relations cachées.

STRATÉGIE B – REFONTE CIBLÉE DES MODÈLES (complète, 1-2 semaines) :

Conserver l'architecture technique (Studio, settings, urls racine).

Réécrire les models.py, views.py, et templates de l'application métier.

IDÉAL SI : Le domaine est fondamentalement différent.

AVANTAGE : Résultat propre, sans résidus du domaine original.

POUR UN NÉOPHYTE : Commencez toujours par la Stratégie A.

Observer comment le système réagit vous enseignera plus que n'importe quel manuel.

8.2 PROCÉDURE DE RENOMMAGE SYSTÉMATIQUE – LES 7 ÉTAPES

CONTEXTE : Transformer Boutique ERP en Gestion Scolaire.

CORRESPONDANCE DES CONCEPTS :

Boutique ERP	→ École Manager
Produit	→ Élève
Categorie	→ Classe
Vente	→ Paiement de frais
LigneDeVente	→ Frais payé
Client	→ Parent/Tuteur
Fournisseur	→ Enseignant
Bon de commande	→ Note/Bulletin

ÉTAPE 1 : Dupliquer le dossier du projet (ne jamais modifier l'original)
`cp -r Boutique_erp/ EcoleManager/`

ÉTAPE 2 : Renommer le dossier de l'application
`mv EcoleManager/boutique/ EcoleManager/ecole/`

ÉTAPE 3 : Chercher & remplacer "boutique" → "ecole" dans settings.py
`INSTALLED_APPS = [..., 'ecole', ...]`

ÉTAPE 4 : Chercher & remplacer dans urls.py racine
`include('ecole.urls')`

ÉTAPE 5 : Dans models.py – renommer les classes et champs
`class Produit → class Eleve`
`class Client → class Tuteur`
`nom, prenom, date_naissance, classe, numero_matricule...`

ÉTAPE 6 : Supprimer toutes les migrations existantes ET la base SQLite
`rm ecole/migrations/0*.py`
`rm db.sqlite3`
→ Puis : `makemigrations + migrate` (repart de zéro)

ÉTAPE 7 : Adapter les templates (renommer les titres, colonnes, labels)
"Liste des Produits" → "Liste des Élèves"
"Prix de vente" → "Frais scolaires"

8.3 POINTS DE VIGILANCE – CE QUI CASSE SOUVENT

PROBLÈME FRÉQUENT	SOLUTION
"No module named 'boutique'"	Vérifier INSTALLED_APPS + import dans tous les fichiers Python
"Table doesn't exist"	Vous avez oublié migrate après makemigrations. Relancer les deux.
"Reverse for 'xxx' not found"	Une URL a été renommée en views.py mais pas dans les templates {% url %}
"OperationalError: no column named 'xxx'"	Un champ existe dans le modèle mais pas en base. Relancer makemigrations
Page blanche / 500 error	Regarder le terminal Studio. Lire la DERNIÈRE ligne du Traceback.

8.4 CHECKLIST DE LIVRAISON – AVANT DE REMETTRE AU CLIENT

- ☐ settings.py : DEBUG = False, SECRET_KEY changé, TIME_ZONE correct
- ☐ Toutes les pages se chargent sans erreur 500
- ☐ Formulaire de création/modification fonctionnent
- ☐ Données de démonstration chargées via fixtures (python manage.py loaddata)
- ☐ Superutilisateur admin créé (bouton Studio ou createsuperuser)
- ☐ Nom de l'organisation visible dans la navbar et les PDF
- ☐ Logo remplacé par celui du client (même nom de fichier)
- ☐ Monnaie correcte affichée (FCFA dans le contexte Congo)
- ☐ backup de la base de données fourni au client (db.sqlite3 + fixtures)
- ☐ Documentation de 1 page fournie : "Comment lancer le système"

8.5 CRÉER DES DONNÉES INITIALES AVEC DES FIXTURES

Les fixtures sont des fichiers JSON qui pré-remplissent la base.

CRÉER UNE FIXTURE à partir de l'état actuel de la base :

```
python manage.py dumpdata ecole.Classe --indent 2 > fixtures/classes_initiales.json
```

CONTENU TYPE D'UNE FIXTURE (fixtures/classes_initiales.json) :

```
[
  {"model": "ecole.classe", "pk": 1, "fields": {"nom": "6ème A", "effectif_max": 45}},
  {"model": "ecole.classe", "pk": 2, "fields": {"nom": "6ème B", "effectif_max": 45}},
  {"model": "ecole.classe", "pk": 3, "fields": {"nom": "5ème A", "effectif_max": 40}}
]
```

CHARGER UNE FIXTURE (chez le client, après migrate) :

```
python manage.py loaddata fixtures/classes_initiales.json
```

→ Le client reçoit un système déjà pré-rempli avec ses propres classes, ses propres matières, ses propres types de frais. Pas une page blanche.

RÉSUMÉ DU CHAPITRE 8

- ✓ Stratégie A (renommage) : rapide, idéale pour domaines similaires
- ✓ Stratégie B (refonte) : propre, pour domaines fondamentalement différents
- ✓ Toujours dupliquer avant de modifier (ne pas altérer l'original)
- ✓ Les 4 erreurs classiques et leurs solutions
- ✓ Checklist de livraison : 10 points à valider avant remise client
- ✓ Fixtures : livrer un système pré-rempli, pas une page blanche

→ SUITE : Chapitre 9 – Déploiement, maintenance et montée en compétence